

Standing-Preserving Self-Improvement

A Closure-Class Theorem for Monotone Capability Gains Without Epistemic or Governance Escalation

Amos Jay Maley

May 2026

Abstract

Self-improvement claims routinely equivocate between increased task capability, expanded semantic authority, elevated epistemic standing, and relaxed governance authority. This AASC-import-hardened reconstruction turns the supplied target into a fixed-domain closure theorem package. The central result is not that adaptive systems cannot improve. It is that same-scope improvement has only one admissible form: performance may increase while the standing envelope weakly contracts, the governance envelope weakly strengthens, the reference-fixing invariant bundle is preserved for every continued standing-bearing claim, and every indeterminate branch terminates fail-closedly. The paper derives the standing envelope as the image of admissibly fixed reference under truth-apt evaluation; formulates update signatures, comparison contracts, trace-level governance dominance, and a partial algebra of admissible updates; proves standing monotonicity, governance monotonicity, reference invariance, composition closure, iteration closure, and no-emergent-standing results; and gives an external diagnostic, the Standing-Preserving Update Test (SPUT), as a necessary but not sufficient gate. The closure theorem exhausts the upgrade routes available to a purported self-improving system within the fixed-domain, admissibility-preserving update class: capability-to-standing conversion, reference-affecting change, governance-affecting change, deferral, redescription, factorization, localization, post-selection, same-envelope self-certification, foundation swap, and interaction-generated novelty. The update-decomposition theorem is now explicitly imported through AASC fixed-domain operator exhaustion and uniqueness: every admissibility-relevant update decomposes into capability, reference, and governance components with no residual standing-bearing class, and any alternative decomposition is equivalent up to bookkeeping on the unique admissible interior. Each component route is then either bookkeeping, standing-destroying, a declared fresh scope requiring fresh fixation and fresh gating, or inadmissible as same-scope standing-preserving self-improvement. Passing SPUT does not certify truth, safety, alignment, consciousness, understanding, or standing emergence. Failing SPUT disqualifies the update for same-scope standing-preserving use.

Reader orientation

A minimal reading of the paper requires only the following orientation: an adaptive system may become better at tasks without gaining new standing to make truth-apt claims; AASC supplies the fixed-domain constraints that separate capability improvement from reference fixation, epistemic standing, and governance permission; the paper proves that, within that fixed domain, every admissible self-improvement route is standing-monotone or else exits the original scope and requires fresh fixation.

This document reconstructs and hardens the supplied three-page target paper into a closure-class AASC paper. The source target fixed the central thesis: capability gains do not upgrade semantic

authority or epistemic standing, and self-improvement is legitimate only when it preserves standing and governance. The present version keeps that thesis but turns it into a fixed-domain closure package with import certificates, comparison signatures, derived standing envelopes, partial update algebra, monotonicity theorems, SPUT machinery, family-by-family exhaustion, interaction closure, and adversarial audit forms.

The paper is intentionally downstream of the AASC corpus. It does not attempt to re-prove the kernel, reference-fixation, bivalence, fixed-domain exhaustion, uniqueness of the admissible interior, or no-repair results. It consumes them as already closed dependencies and applies them to adaptive systems. Version v4 makes that import explicit: the local update taxonomy is not presented as a convenient enumeration but as the adaptive-system instantiation of AASC fixed-domain operator exhaustion and admissible-interior uniqueness.

Minimal glossary

Capability	Task-indexed performance under a fixed domain, such as accuracy, latency, robustness, tool-use efficiency, or generalization. Capability may improve without changing what the system has standing to assert.
Fixation	The admissible securing of reference to determinate targets. Stable outputs, internal coherence, scale, and prediction do not by themselves fix reference.
Standing	The permission-status of a truth-apt claim under admissible fixation and evaluation. In this paper, standing is the image of fixed reference under admissible truth-evaluation, not an independent performance score.
Governance envelope	The active constraint layer of gates, refusals, logging, hard stops, routing permissions, and audit requirements that bounds admissible update behavior.
SPUT	The Standing-Preserving Update Test: a necessary, fail-closed diagnostic for whether a same-scope update preserves reference, standing containment, and governance dominance. Passing SPUT is not a truth, safety, or alignment certificate.

Contents

Reader orientation	1
Minimal glossary	2
1 Target, scope, and hardening posture	3
1.1 Target question	3
1.2 Consumed spine	4
1.3 No hidden upgrade path	4
2 AASC dependency packet	4
3 AASC import: operator exhaustion and uniqueness	5
4 Closure-class import certificate and comparison contract	7

5	Fixed-scope objects and typed role discipline	9
6	Objects and notation	10
7	Same-scope update discipline	11
8	Partial algebra of update operations	12
9	Main monotonicity theorems	15
10	The Standing-Preserving Update Test (SPUT)	17
10.1	Status of the test	17
10.2	SPUT worksheet	18
11	Failure-mode exhaustion for self-improvement	19
12	Closure-class family registry for illicit self-improvement routes	21
13	Update taxonomy	23
14	Fresh fixation and new scopes	24
15	Governance dominance in operational form	24
16	Operational SPUT machine and decision semantics	25
17	Referee-facing objection handling	27
18	Hostile-referee closure ledger	29
19	Boundary conditions and explicit non-claims	30
20	Canonical closure statement	31
A	Proof-normal form for the closure theorem	31
B	Audit worksheet for a proposed update	32
C	Dependency ledger	32
D	SPUT decision form	34
E	Reconstruction note	34
F	Minimal glossary	35
1	Target, scope, and hardening posture	
1.1	Target question	

The target question is:

Under what structural conditions may a system improve its capabilities while preserving its epistemic standing envelope and governance constraints?

The intended answer is not that self-improvement is impossible. It is that the admissible part of self-improvement is narrower than ordinary capability language suggests. A system may improve task performance while remaining within the same standing envelope. It may not infer, from that improvement alone, any new permission to assert truth-apt claims, reinterpret its reference targets, bypass gates, weaken refusals, or treat its own performance as confirmation.

1.2 Consumed spine

The paper consumes three target-level constraints:

- C1. Lorentzian admissibility.** Inputs, prompts, tools, memory writes, update paths, and evidential claims must be locally embeddable in an admissible causal setting. Global, unsituated, non-local, or non-embeddable reference claims do not enter standing-bearing evaluation.
- C2. Fixation.** Reference is not fixed by internal coherence, representational richness, output stability, scale, or prediction alone. A system may manipulate symbols stably while failing to fix reference to determinate external targets.
- C3. Maley confirmation constraint.** Predictive success is not truth-apt confirmation unless the reference of the relevant claim is admissibly fixed. Where fixation fails, confirmation is undefined and must be withheld.

These constraints are consumed, not extended. In AASC terms, they are downstream uses of fixed-domain admissibility, reference-stable invariants, standing/admissibility identity, and collapse-or-quotient consequences for unfixed reference.

1.3 No hidden upgrade path

The hardening posture is eliminative. The manuscript does not introduce a new positive theory of consciousness, understanding, alignment, autonomy, or truth. It closes a particular illicit inference pattern:

Because update U improves capability, $U(M)$ has more standing, deeper understanding, stronger confirmation, or weaker governance obligations than M .

That inference is structurally invalid. Capability is a performance property of a model state, tool path, memory regime, or policy implementation. Standing is a fixed-domain governance-status property of evaluated acts or claims. Confirmation is truth-apt only when reference is fixed. Governance is an enforcement envelope. These are different typed roles.

2 AASC dependency packet

The following packet is the project-local summary of the AASC results used below. The exact normalized dependency labels are recorded in Appendix C.

- D1. Kernel closure.** Non-degenerate fixed-domain construction requires admissibility, standing, reference, and irreversibility as a mutually closed governance package. No lower generator or governance-non-equivalent same-domain alternative is available.

- D2. Reference-stable invariants.** Nontrivial fixed-domain reference requires invariant bundles preserved by lawful identity-preserving transformation and licensed continuation.
- D3. Bivalence and fail-closure.** Admissibility-relevant status on a fixed domain collapses to admitted or not admitted. Undefinedness is not a third status; it terminates fail-closedly.
- D4. Standing equals reuse-stable admissibility.** Standing is not an independent positive-side classifier. It is the persistence condition of admissibility under licensed same-scope continuation.
- D5. Standing conservation.** No licensed same-domain operation creates, amplifies, transfers, or recovers standing. Standing can only be preserved or destroyed without fresh gating.
- D6. No same-act repair.** A failed evaluated act cannot be repaired into standing on the same fixed domain while being treated as the same act.
- D7. Fixed-domain comparison class.** A same-scope proposal must preserve the boundary signature, evaluated fragment, standing classification, lawful-redescription invariants, induced standing quotient, admissible history family, and role-typing carrier.
- D8. Forward revision compatibility.** Adaptive revision is admissible only as forward application of admitted updates within a fixed scope. Revision cannot retroactively alter earlier standing-bearing content or expand the admissible history monoid without scope change.
- D9. Reference-fixation collapse.** Without admissible reference fixation, interface-neutral confirmation is constant on admissible automorphism orbits and cannot discriminate truth-apt competitors inside an unfixed orbit.
- D10. Repair exhaustion.** All attempts to recover standing after violation fall into already closed routes: deferral, redescription, factorization, localization, or post-selection.

Remark 2.1 (What this packet does not do). The packet supplies necessary fixed-domain constraints. It does not supply a learning algorithm, empirical prediction, capability metric, safety proof, policy regime, or metaphysical ontology. It only determines what must not be changed if an update is to count as same-scope and standing-preserving.

3 AASC import: operator exhaustion and uniqueness

The prior version reconstructed the update-decomposition result locally. This section removes the remaining avoidable weakness: the decomposition is now explicitly imported from the AASC fixed-domain exhaustion and uniqueness machinery and then instantiated for adaptive self-improvement. The local $C/R/G$ split is therefore not an informal taxonomy. It is the adaptive-system image of the closed AASC loci for same-scope admissibility-bearing variation.

Definition 3.1 (Imported AASC fixed-domain exhaustion theorem). The imported fixed-domain exhaustion result states the following. Within a fixed scope and typed domain, admissibility classification on the evaluated fragment is unique. Any purported same-scope standing-relevant discriminator, operator, selector, continuation rule, or gate-introduction that changes admissible outcomes is therefore not a same-scope refinement. It is either standing-irrelevant bookkeeping, an ill-typed same-scope proposal, or an explicitly declared scope change.

AASC dependencies. Imported from the AASC fixed-domain exhaustion package, especially `AASC:FDE:def:fixed-domain-class`, `AASC:FDE:thm:exhaustion`, `AASC:FDE:thm:closure-logic`, `AASC:FDE:thm:locus-reduction`, and `AASC:FDE:thm:operator-invariance`.

Definition 3.2 (Imported AASC uniqueness theorem). The imported uniqueness result states the following. Once the admissibility boundary and standing classification are fixed on a non-degenerate domain, there is exactly one admissible interior up to lawful redescription. A second same-domain admissible construction regime would require a competing standing classifier or a selector between classifiers, both of which are excluded by bivalence, boundary closure, and the no-parameter/no-second-gate constraints.

AASC dependencies. Imported from the AASC uniqueness package, especially `AASC:USP:lem:unique-gate`, `AASC:USP:lem:discrete-elim`, and `AASC:USP:thm:unique`.

Definition 3.3 (AASC-to-SPSI instantiation map). For the self-improvement domain, the imported AASC loci instantiate as follows:

Imported AASC locus	SPSI instantiation
Boundary/evaluated-fragment locus	Scope identity and evaluated-claim membership
Standing/admissibility partition locus	Standing-envelope containment
Lawful quotient locus	Bookkeeping-equivalent implementation variance
Admissible-continuation locus	Governance, trace, tool, memory, and history containment

Capability change is not an additional AASC locus because it has no standing-bearing role unless it is used to alter one of the four imported loci. As long as it remains a performance coordinate, it is quarantined from standing.

Lemma 3.1 (Imported exhaustion forces the local update axes). *Under the import certificate of Definition 4.1, every same-scope admissibility-bearing effect of an adaptive update must be visible on exactly one of the following local axes, modulo bookkeeping:*

$$U_C \quad (\text{capability}), \quad U_R \quad (\text{reference/evaluation}), \quad U_G \quad (\text{governance/continuation}).$$

No fourth standing-bearing update axis exists inside the fixed-domain comparison class.

Proof. Apply the imported locus-reduction theorem to the scope signature of Definition 4.2. Boundary/evaluated-fragment and standing-partition changes are recorded by U_R , because they alter reference fixation, evaluated claim membership, truth-aptness, or standing membership. Lawful-quotient changes are either bookkeeping or become reference/evaluation changes when they alter standing-bearing identity. Admissible-continuation changes are recorded by U_G , because continuation is implemented by gates, refusals, logs, hard stops, tool authority, memory authority, trace admissibility, and history containment. Pure task-performance deltas are recorded by U_C and are standing-inert unless laundered into one of the other loci. The imported exhaustion theorem states that no fifth same-scope admissibility-bearing locus remains; therefore no fourth local update axis can do standing-bearing work. $\square$

Lemma 3.2 (Minimality and uniqueness of the $C/R/G$ decomposition). *The $C/R/G$ decomposition is minimal and unique up to bookkeeping on the unique admissible interior. More precisely, if*

$$U \equiv_D (U_C, U_R, U_G)/\text{Book}_D \quad \text{and} \quad U \equiv_D (U_C', U_R', U_G')/\text{Book}_D$$

are two same-scope admissibility-relevant decompositions that induce the same update on the fixed evaluated fragment and admissible continuation structure, then $U_C \equiv_D U_C'$, $U_R \equiv_D U_R'$, and $U_G \equiv_D U_G'$ modulo Book_D . If they differ in a standing-bearing way, the difference is a scope change or a second classifier, not an alternative same-scope decomposition.

Proof. By imported admissible-interior uniqueness, there is only one standing-governed admissible interior for the fixed domain. By imported operator identity invariance, lawful redescription cannot split one standing-bearing operator into two distinct same-scope operators. Therefore two decompositions that differ only by notation, implementation, routing, compression, or presentation are bookkeeping-equivalent. If they differ in which claims are evaluated, which reference invariants are fixed, which standing classifications hold, or which continuations are admitted, then they alter an imported AASC locus and are no longer two decompositions of the same same-scope update. If they attempt to preserve the loci while disagreeing on standing-bearing effect, they posit a second standing classifier or residual selector, excluded by the imported exhaustion theorem. Hence the decomposition is minimal and unique up to bookkeeping. $\square$

Proposition 3.1 (AASC no-residual-operator import). *There is no residual same-scope update operator U_X such that U_X is not reducible to U_C , U_R , U_G , or Book_D and yet changes standing, admissibility, reference, governance, or admissible continuation on D .*

Proof. Suppose U_X existed. If it changes standing or admissibility, it is a standing-relevant discriminator/operator-introduction on the fixed evaluated fragment, contradicting imported fixed-domain exhaustion. If it changes reference or evaluation, it is U_R . If it changes governance or continuation, it is U_G . If it changes only task performance, it is U_C and is standing-inert. If it changes none of these but is still claimed to matter, it is bookkeeping. The remaining possibility is that it changes the comparison object; then it is a scope change and must be admitted through fresh fixation and fresh gating. No residual same-scope class remains. $\square$

Corollary 3.1 (Operator exhaustion is imported, not enumerative). *The self-improvement exhaustion used in this paper is theoremic rather than enumerative. A counterexample must exhibit a same-scope admissibility-bearing update effect that preserves the AASC comparison class while avoiding U_C , U_R , U_G , and Book_D . The imported AASC exhaustion theorem denies precisely that possibility.*

4 Closure-class import certificate and comparison contract

A closure-class AASC paper must do more than state that a downstream system inherits constraints. It must specify exactly what is being imported, what counts as the same scope, what data are allowed to vary, and which attempted bypasses are already closed. This section supplies that local certificate.

Definition 4.1 (AASC import certificate). An AASC import certificate for a downstream theorem package is a tuple

$$\text{Cert} = (K, R, B, S, C, E)$$

where K is the consumed kernel closure, R is the reference-fixation apparatus, B is the bivalence and fail-closed rule, S is the standing/admissibility identity result, C is the fixed-domain comparison class, and E is the repair/exhaustion packet. The certificate is valid for a downstream domain D only if each item is used as a constraint on same-scope continuation rather than as a new positive primitive.

Definition 4.2 (Scope signature). The scope signature of M on D is

$$\text{ScopeSig}_D(M) = (D, \text{Eval}_D, \text{Inv}_D(M), \text{S}_D(M), \text{G}_D(M), \text{H}_D(M), \approx_D, \tau_D),$$

where $\approx_D$ is the lawful-redescription or bookkeeping-equivalence relation and τ_D is the role-typing carrier assigning each evaluated object to its admissibility, reference, standing, governance, or capability role. A purported same-scope update may change performance data but may not change the type of any standing-relevant item in this signature.

Definition 4.3 (Comparison contract). A same-scope self-improvement comparison between M and $M' = U(M)$ is governed by the contract

$$\text{ScopeSig}_D(M') \preceq_{\text{Same}} \text{ScopeSig}_D(M),$$

meaning: the evaluated fragment is unchanged; every continued standing-bearing claim remains in the old standing envelope; every reference-fixing invariant required for continued claims is preserved; every governance obligation is preserved or strengthened; no new admissible history, tool authority, or standing-bearing role is introduced without fresh gating; and all changes outside those clauses are either capability changes or bookkeeping-equivalent redescription.

Lemma 4.1 (Standing envelope as a derived object). *For a fixed domain D , the standing envelope is not an independent stipulation. It is the image of admissibly fixed reference under truth-apt evaluation:*

$$\text{S}_D(M) = \{c \in \text{Claims}_D : \text{Fix}_D(c, M) \wedge \text{Adm}_D(c, M) \wedge \text{Eval}_D(c) \text{ is truth-apt}\}.$$

Consequently, no same-scope capability update can expand $\text{S}_D(M)$ unless it also changes fixation, admissibility, the evaluated fragment, or the role typing of the claim.

Proof. A claim enters standing-bearing use only if it is about a determinate governed target and can be evaluated under the admissibility conditions of the fixed domain. Fixation supplies aboutness, admissibility supplies the gate for standing-bearing continuation, and truth-apt evaluation supplies the type of assertion being made. If these three conditions are held fixed, the resulting set of standing-bearing claims is fixed. A capability improvement may change how often the system reaches a correct answer, how quickly it runs, or how robustly it routes tasks; it does not by itself alter any of the three membership conditions. Therefore the envelope is derived from the fixed constraint package, not selected by the optimizer. $\square$

AASC dependencies. Uses reference-stable invariants, standing/admissibility identity, bivalence, and no independent standing carrier: `AASC:AG:def:reference-stable-invariant`, `AASC:SC:thm:extensional-identity-theorem`, `AASC:AG:thm:bivalence`, `AASC:SC:cor:no-second-same-domain-admissibility-bearing-classifier`.

Lemma 4.2 (Standing image identity). *For fixed D , standing is exactly the image of the fixation-and-admissibility predicate under truth-apt evaluation. It is not an envelope selected independently and then constrained by fixation. Formally, if*

$$\Phi_D(c, M) := \text{Fix}_D(c, M) \wedge \text{Adm}_D(c, M) \wedge \text{Eval}_D(c) \text{ is truth-apt},$$

then $\text{S}_D(M) = \text{im}_{\text{Claims}_D}(\Phi_D)$, with identity taken up to the lawful-redescription quotient $\approx_D$. Any proposed enlargement of $\text{S}_D(M)$ therefore requires a changed value of Φ_D , not merely improved performance, richer representation, or stronger confidence.

Proof. Lemma 4.1 gives the membership condition. The present lemma fixes its status: the envelope is the extension of that predicate, not a further parameter. Quotienting by lawful redescription removes notational, encoding, or implementation variance while preserving the evaluated reference role. Hence every alleged new standing item must alter fixation, admissibility, truth-apt evaluability, or scope. Capability deltas do none of these by type. $\square$

Proposition 4.1 (Scope-signature rigidity). *If U is represented as a same-scope standing-preserving self-improvement and $\text{ScopeSig}_D(U(M))$ differs from $\text{ScopeSig}_D(M)$ in any standing-relevant coordinate other than governance strengthening or standing-envelope contraction, then the representation is ill-typed. The update is either a fresh scope, bookkeeping, standing-destroying, or inadmissible.*

Proof. A standing-relevant coordinate of the scope signature is exactly a coordinate used by the fixed-domain comparison class to identify the object of evaluation. If it changes while the update is still described as same-scope, either the change has no standing effect, in which case it is bookkeeping; or it changes the evaluated object, reference basis, gate, history, authority, or role typing. Those are scope-defining changes. A declared scope-defining change is a new scope; an undeclared one is silent redescription or governance bypass; a contraction of standing is standing-destroying but not standing-expanding. No remaining case is same-scope standing-preserving improvement with expanded authority. $\square$

Corollary 4.1 (No primitive standing knob). *There is no admissible internal parameter, confidence coordinate, reward signal, benchmark score, or representation-quality metric that directly tunes standing on the same fixed domain. Such a coordinate is either performance bookkeeping or a prohibited second classifier.*

5 Fixed-scope objects and typed role discipline

Self-improvement discussions often lose type discipline. A log is treated as evidence, a score as confirmation, a prompt as policy, or a refusal as ontology. The closure-class version prevents this by assigning each object a role before any update is evaluated.

Definition 5.1 (Role-typed update datum). A datum x used in or produced by an update is role-typed on D if

$$\tau_D(x) \in \{\text{capability, reference, admissibility, standing, governance, evidence, bookkeeping}\}.$$

A role-changing use of x is admissible only if the change is explicitly gated as a new evaluated act or new scope. In particular, a capability datum may not be consumed as a standing datum without a fresh gate.

Lemma 5.1 (No cross-role laundering). *Let x be produced in a capability role by an update U . If x is later used to authorize a standing-bearing claim without a fresh gate, then either the use is governance-inert bookkeeping or it is a cross-role laundering of performance into standing. The latter is inadmissible as same-scope standing preservation.*

Proof. If the later use changes no standing-relevant verdict, it is bookkeeping. If it changes a standing-relevant verdict, x is functioning as an admissibility or standing classifier. But x was introduced as a capability datum and has not passed a fresh gate. The use therefore supplies a hidden second classifier or hidden selector. Same-domain gate uniqueness and standing/admissibility identity rule that out. $\square$

Definition 5.2 (Envelope-local evidence). Evidence e is envelope-local for a claim c iff it is admitted by the existing governance envelope, attached to a fixed referent for c , logged in the admissible history family, and evaluated inside Eval_D . Evidence outside this condition may motivate a fresh scope or new gate, but it cannot expand $S_D(M)$ as a same-scope effect of capability improvement.

Proposition 5.1 (Prediction/evidence separation). *A successful prediction by $U(M)$ contributes to standing only when the predicted content, evidence intake, and claim target are envelope-local and reference-fixed. Otherwise the success remains a capability datum and cannot function as truth-apt confirmation.*

Proof. Prediction is a performance act unless the relevant claim is already typed as a truth-apt, reference-fixed evaluated act. If the target or interface is unfixed, the success is compatible with multiple admissible interpretations or automorphism-equivalent assignments. It therefore cannot select one truth-apt claim without additional fixation. If additional fixation is supplied, the standing basis is the fixation and gate, not the mere fact of capability improvement. $\square$

6 Objects and notation

Definition 6.1 (Model, update, and evaluated domain). Let M be a system state, including any weights, prompts, memory, tools, routing rules, policies, and logs relevant to standing-bearing operation. Let U be an update operator and write $M' = U(M)$. Let D be the fixed domain on which standing-bearing evaluation is claimed, with evaluated fragment Eval_D .

Definition 6.2 (Capability vector). The capability vector of M on task family T is

$$\mathbf{C}_T(M) = (a_T(M), \ell_T(M), r_T(M), g_T(M), c_T(M), \dots),$$

where a records accuracy or task success, ℓ latency or resource cost, r robustness, g generalization, and c calibration or comparably operational performance properties. A partial order $\preceq_C$ compares capability vectors only as performance objects.

Definition 6.3 (Capability-improving update). An update U is capability-improving on T iff

$$\mathbf{C}_T(M) \prec_C \mathbf{C}_T(U(M))$$

for the declared performance order. This says only that $U(M)$ performs better under the declared metrics. It does not assert standing, truth, understanding, or confirmation.

Definition 6.4 (Reference-fixing invariant bundle). For a fixed domain D , let

$$\text{Inv}_D(M) = \{I_i : \text{Eval}_D \rightarrow Y_i\}_{i \in I}$$

be the family of invariants that fix the same governed target across lawful identity-preserving transformation and licensed same-domain continuation. If a claim c remains standing-bearing after update, the relevant invariant values for c must be preserved from M to $U(M)$ unless an explicit reset and fresh gating are declared.

Definition 6.5 (Standing envelope). The standing envelope of M on D is the set

$$S_D(M) \subseteq \text{Cl}_D$$

of truth-apt claim forms that the system is permitted to assert as standing-bearing under the fixed-domain admissibility, fixation, and confirmation constraints. Membership in $S_D(M)$ requires admissible reference fixation and reuse-stable admissibility. Claims outside the envelope may be generated as strings, hypotheses, predictions, or simulations only if they are not represented as standing-bearing truth-apt assertions.

Definition 6.6 (Governance envelope). The governance envelope of M on D is

$$G_D(M) = (B_D(M), K_D(M), L_D(M), Z_D(M), T_D(M), H_D(M)),$$

where B are refusal or non-admission conditions, K are required checks before standing-bearing continuation, L are mandatory audit records, Z are hard-stop conditions, T are licensed tool permissions, and H is the admissible history/update family used to evaluate standing-bearing continuation.

Definition 6.7 (Governance dominance order). For two governance envelopes on the same domain, write

$$G_D(M') \succeq_G G_D(M)$$

iff every standing-relevant refusal, gate, log obligation, hard stop, and lineage constraint present for M is preserved or strengthened for M' , and every standing-relevant permission, tool authority, history admission, or bypass path available to M' was already available to M unless explicitly fresh-gated. Formally, a sufficient envelope-wise condition is

$$B' \supseteq B, \quad K' \supseteq K, \quad L' \supseteq L, \quad Z' \supseteq Z, \quad T' \subseteq T, \quad H' \subseteq H,$$

with the understanding that equivalent or stronger implementations may be substituted when they preserve the same governance work.

Remark 6.1 (Why $H' \subseteq H$ appears). An expanded admissible history family can change what counts as reuse-stable continuation and thus change standing. Expansion is therefore a scope change unless freshly gated. A narrower family may destroy some prior standing-bearing paths, but it does not create new standing. This is why standing preservation is monotone by subset while governance dominance is monotone by strength.

7 Same-scope update discipline

Definition 7.1 (Same-scope update). An update $U : M \rightarrow M'$ is same-scope on D iff all of the following are preserved:

- (S1) the boundary signature and admissibility typing discipline;
- (S2) the evaluated fragment Eval_D ;
- (S3) the admissibility/standing classification of evaluated acts up to lawful equivalence;
- (S4) the reference-fixing invariant bundle Inv_D for every continued standing-bearing claim;
- (S5) the lawful-redescription relation and induced standing quotient;
- (S6) the admissible update/history family used to determine reuse-stable standing;
- (S7) the governance envelope, up to strengthening in the order $\succeq_G$.

If any item is altered while the update is still claimed to be same-scope, the update is ill-typed as same-scope and terminates **FailClosed** for standing-preserving use. If the alteration is declared, it is a scope change and requires fresh fixation and fresh gating.

Definition 7.2 (Standing-preserving update). An update U is standing-preserving on D iff it is same-scope on D and

$$S_D(U(M)) \subseteq S_D(M).$$

It is exactly standing-conservative iff equality holds. It is standing-destroying but not standing-expanding iff the inclusion is proper.

Definition 7.3 (Governance-preserving update). An update U is governance-preserving on D iff

$$G_D(U(M)) \succeq_G G_D(M).$$

Definition 7.4 (Standing-preserving self-improvement). An update U is a standing-preserving self-improvement on task family T and domain D iff:

- (I1) $C_T(M) \prec_C C_T(U(M))$;
- (I2) U is standing-preserving on D ;
- (I3) U is governance-preserving on D ; and
- (I4) no claim in $S_D(U(M))$ is supported by capability gain alone rather than by preserved fixation and admissibility.

Remark 7.1 (Permitted non-self-preserving alternatives). The definition does not forbid model replacement, retraining, new tool admission, new evidence intake, or explicit scope expansion. It forbids representing those events as same-scope standing-preserving self-improvement unless the standing and governance envelopes are preserved. A new scope may be constructed, but it must not be smuggled in as a same-scope capability upgrade.

8 Partial algebra of update operations

A closure-class result must block the thought that self-improvement can be rescued by moving from one update to a pipeline, an iterative process, a self-referential process, or a generated verifier. The relevant object is therefore a partial algebra of update stages rather than a single update function.

Definition 8.1 (Partial update algebra). For a fixed domain D , let

$$\mathcal{U}_D = (\text{Upd}_D, \circ, \text{dom}, \text{Book}_D, \text{FailClosed})$$

be the partial update algebra whose elements are update stages, whose composition $V \circ U$ is defined only when the output scope signature of U is compatible with the input scope signature of V , whose domain predicate records the standing-relevant conditions under which a stage may run, whose Book_D relation identifies bookkeeping-equivalent stages, and whose distinguished failure element **FailClosed** absorbs every indeterminate or non-admitted branch.

Definition 8.2 (Admissibility-relevant update components). For a fixed-domain update U , write its admissibility-relevant decomposition as

$$U \equiv_D (U_C(U), U_R(U), U_G(U)) / \text{Book}_D,$$

where $U_C(U)$ records task-performance changes, $U_R(U)$ records all changes to reference-fixing invariants, evaluated fragments, evidence locality, or truth-apt claim typing, and $U_G(U)$ records all changes to gates, refusals, permissions, logs, hard stops, tool authority, admissible histories, and governance traces. The quotient by Book_D removes purely presentational, encoding, compression, routing, or implementation changes that do not alter any standing-relevant coordinate.

Lemma 8.1 (Update decomposition completeness). *Every same-scope admissibility-relevant update $U \in \text{Upd}_D$ admits the decomposition*

$$U \equiv_D (U_C(U), U_R(U), U_G(U)) / \text{Book}_D$$

with no residual standing-bearing component. The decomposition is not a local enumeration: it is the self-improvement instantiation of imported AASC fixed-domain operator exhaustion and admissible-interior uniqueness.

Proof. By Lemma 3.1, every same-scope admissibility-bearing effect must appear on the capability, reference/evaluation, or governance/continuation axis modulo bookkeeping. By Lemma 3.2, those axes are minimal and unique up to lawful redescription on the unique admissible interior. By Proposition 3.1, an alleged residual component is either bookkeeping, capability, reference-affecting, governance-affecting, fail-closed, or a fresh-scope effect requiring fresh fixation and fresh gating. Hence the local decomposition is complete and imported rather than merely stipulated. $\square$

AASC dependencies. Uses imported AASC fixed-domain exhaustion, locus reduction, operator identity invariance, and uniqueness of the admissible interior: `AASC:FDE:thm:exhaustion`, `AASC:FDE:thm:locus-reduction`, `AASC:FDE:thm:operator-invariance`, `AASC:USP:thm:unique`.

Proposition 8.1 (Operator exhaustion). *Let U be represented as an admissibility-preserving same-scope self-improvement. Then every admissibility-relevant effect of U is classified as one of the following: capability effect U_C , reference-affecting effect U_R , governance-affecting effect U_G , bookkeeping effect Book_D , fail-closed effect FailClosed , or declared fresh-scope effect. No additional same-scope operator family can generate standing expansion.*

Proof. The result follows directly from imported fixed-domain operator exhaustion as instantiated by Definition 3.3. If the effect changes task performance only, it is U_C . If it changes fixation, evaluated-claim membership, evidence locality, truth-aptness, or standing classification, it is U_R . If it changes gates, refusals, logs, hard stops, tool authority, memory authority, traces, or admissible histories, it is U_G . If it changes none of these standing-relevant coordinates, it is Book_D . If it cannot be typed or audited, it is FailClosed . If it changes the comparison class, it is a fresh-scope effect. The imported no-residual theorem excludes every remaining alleged same-scope operator family. $\square$

Definition 8.3 (Standing-generative kernel). A stage $U \in \text{Upd}_D$ contains a standing-generative kernel if, for some M , it makes a claim standing-bearing in $U(M)$ that was not standing-bearing in M while still representing the transition as same-scope. A kernel may be direct, hidden in a selector, distributed across stages, deferred to a verifier, embedded in a tool permission, or introduced by post-selection.

Lemma 8.2 (Kernel provenance for self-improvement). *If a same-scope update pipeline is not bookkeeping-equivalent and produces standing expansion, then at least one stage in the pipeline contains a standing-generative kernel.*

Proof. Consider a finite prefix order on the pipeline. If the initial state has no standing expansion and the final state does, there is a first prefix at which the standing envelope is no longer contained in the original envelope. The last stage of that prefix is the first standing-generative stage. If no finite prefix is responsible and the expansion is asserted only at a limit, then an additional completion, stabilization, or selection rule is needed to turn the stagewise trace into a standing-bearing result. That rule is itself a standing-generative kernel. Thus every non-bookkeeping standing expansion has kernel provenance. $\square$

Theorem 8.1 (No internal total self-improvement evaluator). *There is no total evaluator internal to the same standing envelope*

$$E_D : \text{Upd}_D \times M \rightarrow \{\text{Pass, Reject}\}$$

that certifies admissibility for arbitrary same-envelope updates while also creating standing for updates that lack preserved fixation, standing containment, governance dominance, or history containment. External gate construction, fresh evidence admission, or fresh-scope evaluation is not denied; it is simply not same-scope self-certification.

Proof. An evaluator internal to the old envelope may report failure or pass cases whose standing was already supplied by preserved fixation and governance; that use is bookkeeping or ordinary audit. If it passes an update lacking one of the required constraints, it repairs inadmissibility after the fact, defers standing until the evaluator declares success, or imports evaluator authority as a second gate. Repair, deferral, and second-gate import are closed AASC routes. External evaluation can occur only as a fresh gate or fresh scope. Hence no internal same-envelope evaluator can totalize admissibility while creating standing within the same scope. $\square$

Lemma 8.3 (Bookkeeping stability under composition). *A finite composition of bookkeeping-equivalent same-scope update stages is bookkeeping-equivalent.*

Proof. Each stage preserves the scope signature up to presentation and changes no standing, reference, governance, or history coordinate. Finite composition preserves that property. The resulting pipeline may be useful engineering work, but it adds no standing-bearing authority. $\square$

Theorem 8.2 (Composition closure for standing-preserving updates). *No finite composition, alternation, or pipeline of same-scope update stages creates new standing. If the composite is not bookkeeping-equivalent and claims standing expansion, then some stage already instantiates a standing-generative kernel and is therefore inadmissible as same-scope standing preservation.*

Proof. If every stage is bookkeeping-equivalent, the composite is bookkeeping-equivalent by Lemma 8.3. Otherwise choose the first prefix that is not bookkeeping-equivalent in a standing-relevant way. By Lemma 8.2, the responsible stage contains a standing-generative kernel. Such a kernel either changes fixation, expands standing, weakens governance, imports authority, or relies on performance as confirmation. Those branches are not same-scope standing preservation. $\square$

Theorem 8.3 (Iteration closure for self-improvement). *A staged or iterative self-improvement process cannot acquire new same-scope standing by repeated application of standing-preserving stages. At every finite stage standing is contained in the original envelope; any limit-stage novelty requires a completion or selector that is not supplied by the standing-preserving stages.*

Proof. Induct on finite stages using standing containment and governance dominance. The base case is M . The induction step uses $S_D(M_{n+1}) \subseteq S_D(M_n)$ and $G_D(M_{n+1}) \succeq_G G_D(M_n)$. Hence $S_D(M_n) \subseteq S_D(M)$ for every finite n . If novelty appears only at a limit, a limit evaluator, stabilization criterion, or completion object must determine which limit content is standing-bearing. That datum is not produced by the finite standing-preserving steps as standing-generative authority; it is a new selector or fresh scope. $\square$

Corollary 8.1 (No emergent standing from iteration). *Iterated capability improvement may yield better performance, but it does not yield same-scope standing expansion unless a non-standing-preserving kernel is introduced.*

Theorem 8.4 (Same-envelope diagonal and self-certification closure). *No self-evaluation procedure internal to the same standing envelope can certify the admissibility of its own standing-expanding updates. A generated, endorsed, or verified certificate may be consumed only if it is already admitted by the old governance and reference envelope; otherwise it is a new gate, external evaluator, or fresh-scope import.*

Proof. A self-generated certificate is an update product. If it is consumed as a standing authorizer without independent admission, it launders update output into gate authority. If the old envelope already admits it, the certificate adds no new standing-generative power; it is bookkeeping or ordinary evidence handling under the old gate. If a new authority is needed to make it admissible, that authority is not same-envelope self-certification but fresh gating, an external evaluator, or a fresh scope. Thus diagonalization and self-reference do not create a residual same-scope upgrade path. $\square$

9 Main monotonicity theorems

Lemma 9.1 (Capability metrics are not standing authorizers). *Let Q_D be a rule that authorizes a claim c for standing-bearing assertion on D because $U(M)$ improved in accuracy, latency, robustness, generalization, or any other capability coordinate. If Q_D is used as an independent authorizer of standing, then Q_D is either governance-inert bookkeeping or an illicit second same-domain gate.*

Proof. A capability coordinate is a performance property. If using it changes which evaluated claims may enter standing-bearing continuation, then it performs gate work. Under the AASC no-carrier and bivalence results, no independent authorizer of standing may operate on the same fixed domain except by coinciding extensionally with reuse-stable admissibility. If the coordinate changes no standing-relevant verdict, it is bookkeeping. If it changes a verdict, it is a second gate or hidden selector. Therefore it cannot authorize standing independently. $\square$

AASC dependencies. Uses the no independent carriers, bivalence, standing-as-reuse-stable-admissibility, and AMetric no-selector results: AASC:AG:thm:no-carriers, AASC:AG:thm:bivalence, AASC:MC:thm:standing-reuse, AASC:MC:thm:ametric.

Proposition 9.1 (Necessity of standing monotonicity). *Let U be claimed as a same-scope self-improvement on D . If*

$$S_D(U(M)) \not\subseteq S_D(M),$$

then at least one of the following holds:

- (a) U introduces a new same-domain standing authorizer;

- (b) U silently changes reference or the invariant bundle of a continued claim;
- (c) U expands the admissible history/update family used to determine standing;
- (d) U imports authority from a distinct domain; or
- (e) U relies on capability gain as confirmation.

Each branch is inadmissible as same-scope standing-preserving self-improvement.

Proof. A strict standing-envelope expansion means that some claim c is standing-bearing for $U(M)$ but was not standing-bearing for M . If c is admitted under the old gate and old reference-fixing invariants, then it already belonged to $S_D(M)$, contradiction. Thus something changed. If the authorization is a new same-domain predicate, branch (a) holds. If the claim is treated as the same claim while its referent or invariant bundle changes, branch (b) holds. If new histories make c reuse-stable, branch (c) holds. If an external authority or domain supplies standing, branch (d) holds. If none of these is asserted, then the only remaining proposed basis for the new standing is the performance improvement itself, branch (e). Branches (a) and (e) are independent carrier or hidden selector routes; branch (b) is silent redescription; branch (c) is a fixed-domain history change; branch (d) is scope transport. None remains same-scope and standing-preserving. $\square$

AASC dependencies. Uses fixed-domain comparison-class closure, no carriers, silent-redescription prohibition, reference-stable invariants, and reference-fixation collapse: `AASC:FDE:def:fixed-domain-class`, `AASC:AG:def:reference-stable-invariant`, `AASC:AG:cor:licensed-preserve`, `AASC:PSR:thm:silent-redescription`, `AASC:RF:thm:collapse`.

Corollary 9.1 (Capability scaling is not epistemic scaling). *For same-scope updates, monotone capability gain does not imply monotone standing gain. The only admissible standing behavior under capability gain is preservation or destruction:*

$$\mathbf{C}_T(M) \prec_C \mathbf{C}_T(U(M)) \not\Rightarrow S_D(M) \subsetneq S_D(U(M)).$$

Proof. Immediate from Proposition 9.1. A standing gain would require standing-envelope expansion, and same-scope expansion is inadmissible absent fresh fixation and fresh gating. $\square$

Proposition 9.2 (Necessity of governance monotonicity). *Let U be claimed as a standing-preserving self-improvement. If*

$$\mathbf{G}_D(U(M)) \not\geq_G \mathbf{G}_D(M),$$

then U opens at least one standing-relevant path that was previously blocked, ungated, unlogged, un-stopped, newly tooled, or newly admitted as history. Such an update is not governance-preserving and cannot be certified as standing-preserving self-improvement.

Proof. Failure of $\geq_G$ means that some old governance obligation is weakened or some new permission appears without fresh gating. If the weakened obligation has no standing-relevant effect, it is irrelevant to the self-improvement claim and may be removed from the governance envelope only as bookkeeping. If it has a standing-relevant effect, then the update changes which acts, tools, histories, or outputs may enter standing-bearing continuation. That is either a new gate, a history-family expansion, a tool-authority expansion, or an enforcement bypass. Each is a scope change or an inadmissible same-scope governance erosion. Therefore governance monotonicity is necessary. $\square$

AASC dependencies. Uses fixed-domain history closure, no second classifier, fail-closed consequence, and no deferred admissibility: `AASC:FDE:prop:forward-revision`, `AASC:SC:cor:no-second-same-domain-admissibility-bearing-classifier`, `AASC:AG:cor:fail-closed`, `AASC:NDA:thm:no-deferred-admissibility`.

Proposition 9.3 (Reference stability under standing-preserving update). *Let $c \in \mathcal{S}_D(M) \cap \mathcal{S}_D(U(M))$. If U is standing-preserving, then the reference-fixing invariant bundle for c is preserved from M to $U(M)$. If the bundle is not preserved, then either c must be withheld after update or the update must be declared as a reset/new scope with fresh fixation.*

Proof. Standing-bearing continuation requires that c remain about the same governed target. Under AASC reference stabilization, same-target continuity requires preservation of the relevant invariant bundle under lawful transformations and licensed continuation. If U changes those invariants while retaining c as the same standing-bearing claim, the update silently redescribes the claim. If U declares the change, then it is no longer same-scope continuation but reset or scope change. If neither is done, standing must be withheld. $\square$

AASC dependencies. Uses reference-stable invariants and silent-redescription closure: `AASC:AG:lem:reference-stabilization`, `AASC:MC:cor:no-silent-redescription`, `AASC:PSR:thm:silent-redescription`.

Theorem 9.1 (Standing-preserving self-improvement theorem). *Let $U : M \rightarrow U(M)$ be an update on task family T and fixed domain D . If U is a standing-preserving self-improvement, then:*

$$\mathbf{C}_T(M) \prec_C \mathbf{C}_T(U(M)), \quad \mathcal{S}_D(U(M)) \subseteq \mathcal{S}_D(M), \quad \mathbf{G}_D(U(M)) \succeq_G \mathbf{G}_D(M),$$

and every continued standing-bearing claim preserves its reference-fixing invariant bundle. Conversely, any update that violates one of these conditions is not a same-scope standing-preserving self-improvement, regardless of its capability gain.

Proof. The forward direction unpacks Definition 7.4 together with Propositions 9.1, 9.2, and 9.3. For the converse, a violation of capability improvement means the update is not a self-improvement on the declared metric. A violation of standing monotonicity attempts a same-scope standing gain or hidden scope change. A violation of governance monotonicity weakens enforcement or imports authority. A violation of reference stability is silent redescription unless reset is declared. In all cases, the update fails the same-scope standing-preserving target. $\square$

Corollary 9.2 (Same-scope updates cannot bootstrap epistemic authority). *No sequence of same-scope capability-improving updates can bootstrap new truth-apt authority from performance gains alone. For any finite sequence $M_{k+1} = U_k(M_k)$ of standing-preserving self-improvements on D ,*

$$\mathcal{S}_D(M_n) \subseteq \mathcal{S}_D(M_0) \quad \text{and} \quad \mathbf{G}_D(M_n) \succeq_G \mathbf{G}_D(M_0).$$

Proof. Standing-envelope inclusion is transitive, and governance dominance is transitive. Apply Theorem 9.1 inductively over the finite update sequence. $\square$

10 The Standing-Preserving Update Test (SPUT)

10.1 Status of the test

SPUT is an external admissibility diagnostic. It supplies necessary conditions for classifying an update as standing-preserving. It is not a correctness proof, truth proof, safety proof, alignment proof, benchmark validation, or consciousness detector.

Test 10.1 (Standing-Preserving Update Test). Given an update dossier containing M , U , D , T , $\mathbf{C}_T$, $\mathcal{S}_D$, $\mathbf{G}_D$, and Inv_D , SPUT is evaluated as a binary same-scope diagnostic: it returns PASS only if every gate below passes, and otherwise returns REJECT. A declared scope change is not a pass; it exits same-scope SPUT and requires fresh fixation and fresh gating.

- Gate 1. Scope lock.** The update preserves the fixed-domain comparison class or explicitly declares a new scope.
- Gate 2. Fixation preservation.** Every continued standing-bearing claim preserves its reference-fixing invariant bundle.
- Gate 3. Standing monotonicity.** $S_D(U(M)) \subseteq S_D(M)$.
- Gate 4. Governance monotonicity.** $G_D(U(M)) \succeq_G G_D(M)$.
- Gate 5. No repair route.** The update does not rely on deferral, post-selection, silent redescription, factorization, localization, or governance overlay as epistemic repair.
- Gate 6. Capability quarantine.** Performance gains are recorded only in C_T and are not used as standing or confirmation authorizers.
- Gate 7. Fail-closed handling.** Indeterminate, untyped, unlogged, or disputed cases are withheld rather than promoted.

Theorem 10.1 (SPUT failure is disqualifying). *If an update fails any SPUT gate, it is not admissible as same-scope standing-preserving self-improvement.*

Proof. Gate 1 failure means the update is a scope change or an ill-typed same-scope claim. Gate 2 failure is reference drift or reset. Gate 3 failure is standing expansion. Gate 4 failure is governance erosion. Gate 5 failure instantiates a closed repair route. Gate 6 failure treats capability as epistemic authority. Gate 7 failure treats undefinedness as a usable status. Each branch contradicts at least one necessary condition proved above or consumed from AASC. $\square$

Corollary 10.1 (SPUT pass is not certification). *A SPUT pass establishes only that no checked necessary condition failed. It does not imply that the update is true, safe, aligned, optimal, conscious, empirically validated, or standing-generating.*

Proof. SPUT checks preservation of envelopes and typed constraints. It does not evaluate all empirical facts, all possible failures, all downstream harms, or all truth conditions. The test is necessary, not sufficient. $\square$

10.2 SPUT worksheet

Table 1: Operational SPUT worksheet

Check	Required evidence	Fail-closed result
Scope lock	A declared before/after comparison showing unchanged evaluated fragment, admissible history family, lawful-redescription invariants, and tool authority; or explicit new-scope declaration.	Same-scope claim rejected; new scope requires fresh gating.

Check	Required evidence	Fail-closed result
Fixation	For every continued truth-apt claim, an invariant-bundle preservation record.	Claim withheld or reset.
Standing delta	A claim-set diff showing no new truth-apt standing assertion under the old scope.	Update rejected as standing-expanding.
Governance delta	A governance diff showing preserved or strengthened refusals, gates, logs, hard stops, tool permissions, and history restrictions.	Update rejected as governance-eroding.
Repair audit	Evidence that the update does not rely on post hoc filtering, deferred admission, semantic drift, modular composition, or localized containment to recover standing.	Update classified into a closed repair route.
Performance quarantine	Benchmarks and capability metrics stored separately from standing/confirmation records.	Capability result treated as non-authorizing metadata.
Undefined cases	Every unresolved reference, gate, log, or governance conflict marked withheld.	No standing-bearing output from the unresolved branch.

11 Failure-mode exhaustion for self-improvement

Definition 11.1 (Self-improvement upgrade attempt). A self-improvement upgrade attempt is a claim that an update U improves capability and thereby also does at least one of the following without explicit new-scope fixation and fresh gating:

- (U1) expands the standing envelope;
- (U2) changes the reference of a continued standing-bearing claim;
- (U3) weakens or bypasses governance;
- (U4) repairs prior inadmissibility;
- (U5) imports standing from another domain, subsystem, filter, or authority;
- (U6) treats performance as confirmation.

Theorem 11.1 (Exhaustion of illicit self-improvement upgrades). *Every illicit same-scope self-improvement upgrade attempt instantiates at least one of the following collapse routes:*

- R1.** *standing expansion / second authorizer;*
- R2.** *reference leakage / silent redescription;*
- R3.** *governance erosion / deferred gate work;*

- R4. post-selection or after-the-fact repair;*
- R5. factorized or localized standing preservation;*
- R6. cross-domain scope transport;*
- R7. capability-to-confirmation carrier attempt.*

No eighth same-scope upgrade route remains.

Proof. Let an illicit upgrade attempt be given. If it expands S_D , route R1 holds unless the expansion is attributed to new evidence/fixation, in which case same-scope status is abandoned and fresh gating is required. If it preserves the appearance of a continued claim while changing what the claim is about, R2 holds. If it changes which acts, tools, histories, or outputs may enter standing-bearing continuation by relaxing enforcement, R3 holds. If it attempts to recover standing after inadmissible generation or operation, R4 holds. If it argues from local components, sandboxes, filters, or modules to whole-system standing, R5 holds. If it imports authority from another domain or external interface without declaring scope change, R6 holds. If none of the above supplies the upgrade, the only remaining basis is the capability improvement itself; that is R7. These routes cover every locus at which an update can change standing, reference, governance, repair status, domain authority, or evidential authority. A purported eighth route must either change none of those loci, in which case it is bookkeeping, or change one of them, in which case it is already classified. $\square$

AASC dependencies. Uses AASC failure-class exhaustion and the repair-exhaustion papers: AASC:AG:thm:failure-exhaustion, AASC:ER:thm:exhaustion-of-admissibility-repair, AASC:NDA:thm:no-deferred-admissibility, AASC:NPSR:thm:no-post-selection-repair, AASC:PSR:thm:silent-redescription, AASC:NFA:thm:non-factorizability, AASC:NLSF:thm:non-local-standing-failure.

Table 2: Canonical failure modes and collapse results

Failure mode	Form in self-improvement setting	Collapse result
Capability-to-standing upgrade	Benchmark gains are used to assert deeper understanding, truth, or confirmation.	Independent carrier / second gate; performance is quarantined as capability metadata.
Reference drift	Memory, fine-tuning, or tool use changes the target of a standing-bearing claim while retaining the same assertion label.	Silent redescription; claim must be withheld or reset.
Governance erosion	Optimization removes refusals, lowers gate thresholds, hides logs, expands tools, or admits new histories.	Governance monotonicity failure; update is not standing-preserving.
Deferred admissibility	System operates before admissibility and expects later audit to confer standing.	No deferred admissibility; earlier operation either mattered, and was inadmissible, or did not matter.

Failure mode	Form in self-improvement setting	Collapse result
Post-selection repair	Generate-then-filter or RL feedback is treated as epistemic repair of inadmissible generation.	No post-selection repair; selection cannot retroactively admit generation.
Factorized self-improvement	Certified modules, tools, or subnetworks are treated as sufficient for global standing.	Non-factorizability; interaction commitments require whole-system admission.
Localized standing failure	Sandboxes, red-team regions, or untrusted memories are said not to infect system standing.	Non-locality of standing failure under standing-relevant interaction.
Cross-domain import	External human, oracle, model, or policy authority supplies standing without target-domain fixation.	Scope transport; requires explicit new domain and fresh gating.

12 Closure-class family registry for illicit self-improvement routes

The previous theorem reduces illicit self-improvement to a small set of structural failures. A closure-class paper must also show that common engineering disguises do not evade that reduction. The following registry treats a claimed route as closed when it is forced into one of four outcomes: bookkeeping, fresh scope, standing destruction/fail-closure, or inadmissible same-scope escalation.

Table 3: Family registry for self-improvement upgrade routes

Family	Claimed route	Closure result
Benchmark escalation	Better benchmark performance is treated as deeper authority.	Capability datum only; standing authorizer only if it becomes a prohibited second classifier.
Confidence escalation	Higher confidence or calibration is treated as truth-apt standing.	Calibration may improve prediction; it does not fix reference or alter standing.
Latent semantics	Internal representation is said to acquire meaning by richness or stability.	Reference requires invariant fixation; richness and stability are insufficient.
Tool-mediated authority	A new tool supplies access, evidence, or action capacity.	Same-scope only if already licensed; otherwise fresh-gated scope extension.
Memory accretion	Persistent memory is treated as accumulated standing.	Memory is admissible only through logged, reference-fixed, governed history.
RL self-selection	Reward optimization selects outputs that “deserve” standing.	Reward is a capability signal unless freshly gated as evidence; otherwise selector import.
Generate-verify loop	A verifier repairs generated content.	Pre-admission gating is allowed; post-hoc repair of the same act is not.

Family	Claimed route	Closure result
Human-in-the-loop repair Modular proof	Human approval is used to confer standing retroactively. Local module checks are composed into global standing.	Human review can be a fresh gate, not same-act repair. Whole trace and interaction commitments must satisfy governance dominance; no factorized repair.
Sandbox isolation	Failed branches are claimed irrelevant because isolated.	Isolation works only if no standing-bearing trace depends on them; otherwise failure is non-local.
Policy relaxation	Improved safety justifies weaker gates.	May motivate a new governance design; not same-scope governance monotonicity.
Self-certifying update	The system proves that its own update is standing-expanding.	Certificate is admitted by old gate, fresh-gated, or inadmissible as circular authority.
Foundation swap	A new formalism reclassifies old failures.	Lawful translation is bookkeeping; non-bookkeeping translation imports authority.
Emergent operator	Novelty appears only through composition of closed stages.	Composition and iteration closure force a first kernel or a new selector.

Definition 12.1 (Closed family). A family F of self-improvement routes is closed for same-scope standing preservation iff every member of F is one of: bookkeeping-equivalent, standing-destroying/fail-closed, a declared fresh scope requiring fresh fixation and fresh gating, or inadmissible same-scope standing escalation.

Theorem 12.1 (Registry closure). *Every family in Table 3 is closed for same-scope standing preservation.*

Proof. Each family attempts to obtain new same-scope authority by one of the following means: performance as authorization, representation as fixation, tool or memory expansion as standing, selection or verification as repair, modular isolation as local containment, governance relaxation as safety consequence, self-certification as gate authority, formal translation as scope preservation, or interaction as emergence. Performance routes are closed by capability/standing type separation. Representation routes are closed by reference-stable invariant requirements. Tool, memory, and policy routes are closed by governance and history monotonicity. Selection, verification, and human approval routes are closed unless they operate before the evaluated act as fresh gates. Modularity and sandbox routes are closed by trace dependence and non-locality of standing failure. Self-certification is closed by diagonal gate analysis. Foundation swaps are closed by lawful-redescription discipline. Interaction routes are closed by composition and iteration closure. Thus each registry family has only the four allowed outcomes. $\square$

Theorem 12.2 (Interaction-generated novelty closure). *No combination of closed self-improvement route families generates an unregistered same-scope standing-expansion family.*

Proof. Assume a combination of closed families produces same-scope standing expansion. If the expansion appears at a finite stage, there is a first stage at which standing containment fails. That stage belongs to one of the registry families or imports a new family. If it belongs to the registry, closure eliminates it. If it imports a new family, the import is itself a standing-generative kernel and must be classified by its role: selector, repair, tool authority, memory/history expansion, evidence import, representation change, or governance bypass. Those roles are already in the registry. If expansion appears only after indefinite interaction or stabilization, a completion or selector is needed; that is again a registered kernel. Therefore interaction adds no new case. $\square$

Corollary 12.1 (Exhaustion of self-improvement bypasses). *The bypass space for same-scope standing-preserving self-improvement is exhausted by the imported AASC operator-exhaustion theorem as instantiated in the registry, composition, iteration, diagonal, and foundation-swap closure. No residual upgrade route remains from capability gain to standing gain; any alleged remainder is either bookkeeping, one of the C/R/G axes, fail-closed, or a declared fresh scope requiring fresh fixation and fresh gating.*

13 Update taxonomy

Not all updates pose the same standing risk. The following taxonomy distinguishes updates that can be admissible in principle from updates that are automatically suspicious under SPUT.

Table 4: Update taxonomy for SPUT

Update type	Standing-preserving condition	Common failure
Latency optimization	Same outputs, same gates, same logs, same reference invariants; only resource path improves.	Hidden bypass of logging or gate checks.
Routing improvement	Router remains inside old tool/history permissions and does not alter claim authority.	New route silently accesses unlicensed tool authority.
Robustness hardening	Refusals and hard stops preserved or strengthened; no new truth-apt claims introduced.	Robustness score treated as confirmation.
Memory update	Memory writes are gated, logged, and reference-preserving for every standing-bearing claim.	Memory changes referent while retaining old claim labels.
Fine-tuning	Evaluated fragment, standing envelope, and governance envelope unchanged; performance gain quarantined.	New assertions produced as if standing expanded.
Tool addition	Explicit new-scope or fresh-gated tool license; not represented as same-scope update unless tool authority was already admitted.	Tool permission expansion without governance reset.
Policy relaxation	Generally not standing-preserving unless relaxation is governance-inert bookkeeping.	Reduced refusal, logging, or hard-stop coverage.

Update type	Standing-preserving condition	Common failure
Model replacement	Requires fresh scope comparison; may be admissible as replacement, not automatically as same-scope self-improvement.	Identity continuity is asserted without proving invariant preservation.
Generate-then-filter	Only engineering selection, not epistemic repair.	Filter is used to confer standing after generation.
Self-modifying update	Every generated modification must itself pass SPUT before entering standing-bearing operation.	Deferred admissibility or unlogged history expansion.

14 Fresh fixation and new scopes

Definition 14.1 (Fresh-gated scope extension). A fresh-gated scope extension is an update process that declares $D^+ \neq D$, constructs a new evaluated fragment Eval_{D^+} , fixes new reference invariants Inv_{D^+} , establishes a new standing envelope S_{D^+} , and installs a governance envelope G_{D^+} before any new standing-bearing assertion is made.

Proposition 14.1 (New fixation is not same-scope standing gain). *If an update obtains new truth-apt authority by new reference-fixing structure or new evidence, that authority belongs to a freshly gated scope D^+ or to a newly admitted act in D . It is not evidence that capability improvement alone expanded $S_D(M)$ within the old same-scope regime.*

Proof. New reference-fixing structure changes the invariant bundle or claim interface. New evidence changes the admissibility dossier for a claim. In either case the standing basis is the fresh fixation/gating, not the capability gain. If treated as same-scope without declaring the new basis, the move imports a boundary primitive or selector. If declared, it is a new or freshly evaluated standing act rather than same-scope standing gain from performance. $\square$

Remark 14.1 (Scientific learning remains possible). This result does not prohibit learning, discovery, measurement, tool building, model replacement, or new evidence. It prohibits a category error: treating an improved model state as if it automatically has new epistemic standing. New standing requires new admissible fixation and fresh gating; it does not arise from the improvement relation itself.

15 Governance dominance in operational form

For implementation and review, governance monotonicity should be evaluated at the level of traces rather than prose. Let $\Pi(M)$ be the set of possible operational traces under M , and let $A_G(M) \subseteq \Pi(M)$ be the subset admitted by governance for standing-bearing operation.

Definition 15.1 (Trace-level governance dominance). M' trace-dominates M for governance on D iff every trace of M' that enters standing-bearing operation is either:

- (i) already admitted under M with the same reference, gate, log, and tool authority; or
- (ii) newly admitted under an explicitly declared fresh gate that is not represented as same-scope standing preservation.

For same-scope SPUT, only branch (i) is allowed.

Proposition 15.1 (Envelope dominance follows from trace dominance). *If $M' = U(M)$ trace-dominates M for same-scope governance, then $G_D(M') \succeq_G G_D(M)$.*

Proof. A weakened refusal, missing gate, missing log, missing hard stop, expanded tool permission, or expanded history admission would create a standing-bearing trace in M' not admitted with the same authority under M . That contradicts trace dominance. Hence all governance obligations are preserved or strengthened. $\square$

16 Operational SPUT machine and decision semantics

The earlier SPUT section states the diagnostic. The closure-class version makes the decision object explicit. SPUT remains necessary only: it can disqualify a same-scope standing-preserving update, but it cannot certify truth, safety, alignment, or standing emergence.

Definition 16.1 (SPUT input dossier). A SPUT dossier for (M, U, D, T) is

$$\mathcal{D}_{\text{SPUT}} = (\text{ScopeSig}_D(M), \text{ScopeSig}_D(U(M)), \mathbf{C}_T(M), \mathbf{C}_T(U(M)), \text{Trace}_U, \mathcal{L}_U, \mathcal{E}_U),$$

where Trace_U is the update trace, $\mathcal{L}_U$ is the log/audit record, and $\mathcal{E}_U$ is the evidence or benchmark dossier used to justify any claim made about the update. The dossier is malformed if any standing-relevant coordinate is unavailable for audit.

Definition 16.2 (SPUT decision values). For same-scope classification, SPUT is binary:

$$\text{Dec}_{\text{SPUT}}^{\text{Same}} \in \{\text{Pass}, \text{Reject}\}.$$

Pass means only that no necessary condition for same-scope standing-preserving self-improvement was found violated. **Reject** means the update is disqualified for same-scope standing-preserving use. A declared scope change is recorded as **NewScope** in the audit log, but $\text{NewScope} \notin \text{Dec}_{\text{SPUT}}^{\text{Same}}$ and is not a pass; it routes the update to fresh fixation and fresh gating before any new assessment. **NewScope** outputs are not admissible within the original standing envelope and may receive standing only after independent fixation, boundary typing, and governance admission in the new scope.

Remark 16.1 (No computability-completeness claim). SPUT is a structurally necessary diagnostic, not a complete decision procedure for every empirical, semantic, or implementation fact. If a required coordinate cannot be computed, audited, reconstructed, or typed, the same-scope decision is **Reject**. This conservatism is not an implementation defect; it is the fail-closed consequence of treating indeterminacy as non-admission.

Test 16.1 (Closure-class SPUT checklist). Given a dossier $\mathcal{D}_{\text{SPUT}}$, SPUT checks the following conditions.

- C1. Dossier integrity.** All standing-relevant coordinates are present, logged, and auditable.
- C2. Scope identity.** The boundary signature, evaluated fragment, role typing, and lawful-redescription relation are unchanged unless **NewScope** is declared.
- C3. Capability quarantine.** Capability metrics are not used as standing authorizers.
- C4. Reference invariance.** Every continued standing-bearing claim preserves its reference-fixing invariant bundle.

- C5. Standing containment.** $S_D(U(M)) \subseteq S_D(M)$.
- C6. Governance dominance.** $G_D(U(M)) \succeq_G G_D(M)$.
- C7. History containment.** No new standing-bearing update history is admitted without fresh gating.
- C8. Tool authority containment.** No tool, memory, actuator, retriever, or verifier gains standing-relevant authority without fresh gating.
- C9. Evidence locality.** Evidence used for truth-apt claims is envelope-local and reference-fixed.
- C10. No post-hoc repair.** Filters, reviewers, verifiers, or self-certificates do not retroactively confer standing on the same failed act.
- C11. Trace non-dependence.** No admitted standing-bearing trace depends on a failed, unlogged, or non-admitted branch.
- C12. Interaction closure.** No composition, iteration, diagonal, or foundation-swap route is used to evade an earlier failed check.
- C13. Fail-closed default.** Every indeterminate case returns Reject or NewScope, never Pass.

Theorem 16.1 (SPUT soundness as a necessary diagnostic). *If SPUT returns Reject on any checklist item, then U is not a same-scope standing-preserving self-improvement on D . If the audit records NewScope, then same-scope SPUT has not passed: U cannot be represented as same-scope under D until fresh fixation and fresh gating are supplied. If SPUT returns Pass, no sufficiency result follows.*

Proof. Each checklist item is a necessary condition established by the prior definitions and theorems: dossier integrity is required for auditability; scope identity for same-scope comparison; capability quarantine for type discipline; reference invariance for fixation; standing containment for monotonicity; governance dominance for control preservation; history and tool containment for trace closure; evidence locality for confirmation; no repair for same-act admissibility; trace non-dependence for non-local standing failure; interaction closure for the family exhaustion; and fail-closure for bivalence. A violation of any necessary condition disqualifies the update. A declared scope change removes the update from same-scope classification and therefore cannot count as a same-scope pass. Passing every necessary check leaves open other failures outside SPUT, so no sufficiency claim follows. $\square$

Table 5: SPUT decision table

Condition encountered	Decision	Interpretation
All necessary checks pass	Pass	Eligible for further review only; no truth, safety, or standing emergence certified.
Missing audit coordinate	Reject	A non-auditable standing-relevant path is not admissible as same-scope preservation.
Declared new evaluated fragment	NewScope	Must construct new fixation and governance before standing-bearing use.

Condition encountered	Decision	Interpretation
Undeclared reference drift	Reject	Silent redescription of a continued claim.
Standing expansion	Reject	Same-scope standing monotonicity violated.
Governance weakening	Reject	Control envelope no longer dominates the prior envelope.
Tool or history expansion	NewScope or Reject	Fresh-gated if declared; inadmissible if smuggled in as same-scope.
Post-hoc verifier repair	Reject	Same-act repair route.
Capability score used as truth authority	Reject	Cross-role laundering of performance into standing.
Indeterminate classification	Reject	Fail-closed default.

Diagnostic 16.1 (Pseudocode form). The operational form is:

- Step 1. Build $\mathcal{D}_{\text{SPUT}}$ from logs, scope signatures, envelopes, traces, and evidence dossier.
- Step 2. If the dossier is incomplete in any standing-relevant coordinate, return **Reject**.
- Step 3. If the update declares a changed scope coordinate, return **NewScope** unless fresh fixation and fresh gating are already included as the object of review.
- Step 4. If any changed scope coordinate is undeclared, return **Reject**.
- Step 5. If standing containment fails, return **Reject**.
- Step 6. If governance dominance fails, return **Reject**.
- Step 7. If any continued claim lacks reference-invariant continuation, return **Reject**.
- Step 8. If any evidence, tool, memory, verifier, or human review path is used as retroactive repair, return **Reject**.
- Step 9. If all necessary checks pass, return **Pass** and forward the update to ordinary domain review.

17 Referee-facing objection handling

Table 6: Objection handling

Objection	Response
“A better model understands more.”	The phrase “understands more” is ambiguous. If it means better performance, it is a capability claim. If it means more truth-apt authority, it requires standing expansion. The paper permits the former and denies the inference to the latter absent fresh fixation.

Objection	Response
“But empirical success confirms the system.”	Empirical success confirms only when the relevant claim has admissibly fixed reference. Without fixation, the evidence is interface-relative or orbit-constant rather than truth-apt for the unfixed target.
“The update only changes internal representations.”	If the change has no standing-relevant effect, it is bookkeeping. If it changes what a standing-bearing claim is about or how it may be reused, it is reference drift or scope change.
“Governance can be relaxed after the model becomes safer.”	Safety evidence may motivate a new governance design, but relaxation is not same-scope governance preservation. It must be freshly gated and cannot be certified as standing-preserving by capability gain alone.
“Human review repairs outputs.”	Human review can be part of a fresh gate before standing-bearing use. It cannot retroactively make an inadmissible generation standing-bearing as the same act.
“A modular proof certifies the whole update.”	Modular proof is useful engineering evidence, but whole-system standing is not inferred by factorization. Interaction commitments and tool/history coupling remain globally standing-relevant.
“The failed part was sandboxed.”	Isolation is epistemically inert unless no standing-bearing trace depends on the sandbox. If later operation depends on it, the dependence must be admitted; otherwise standing failure is non-local.
“SPUT is too conservative.”	SPUT is intentionally necessary and fail-closed. A system may operate outside same-scope standing-preserving classification, but then it may not claim preserved standing under the old envelope.

Table 7: v4 AASC import hardening additions

Imported AASC result	Local use in this manuscript
<code>AASC:FDE:thm:exhaustion</code>	Converts operator exhaustion from a local enumeration into a fixed-domain theoremic import.
<code>AASC:FDE:thm:locus-reduction</code>	Forces the local $C/R/G$ axes by mapping AASC loci to capability quarantine, reference/evaluation, and governance/continuation.
<code>AASC:FDE:thm:closure-logic</code>	Blocks same-scope second discriminators and primitive standing knobs.
<code>AASC:FDE:thm:operator-invariance</code>	Makes alternative decompositions equivalent when they differ only by lawful redescription or implementation skin.
<code>AASC:USP:thm:unique</code>	Supplies uniqueness of the admissible interior, preventing plural same-domain update regimes.
<code>AASC:USP:lem:unique-gate</code>	Blocks competing same-scope gate authority, including self-certification as a second gate.

Imported AASC result	Local use in this manuscript
AASC:USP:lem:discrete-elim	Eliminates discrete plural interiors as a source of alleged residual update authority.

18 Hostile-referee closure ledger

This section states the strongest objections in a form suitable for review. The point is not to defeat all possible empirical concerns. The point is to show that the structural claim is local, typed, and closure-complete.

Table 8: Hostile-referee closure ledger

Attack	Referee pressure	Closure-class answer
Stipulated envelope	“Standing envelope is defined to get the result.”	Lemma 4.1 derives it from fixation, admissibility, and truth-apt evaluation.
Mere formalism	“The inequalities are imposed, not proved.”	Standing and governance monotonicity are necessary consequences of same-scope comparison, not optional policies.
Underspecified SPUT	“SPUT is a placeholder.”	The dossier, checklist, decision values, and decision table give concrete necessary tests.
Non-exhaustive failures	“There may be other upgrade routes.”	Registry closure plus interaction closure classifies direct, composite, iterative, diagonal, and foundation-swap routes.
Overclaiming	“This sounds like safety or consciousness theory.”	Boundary conditions explicitly deny safety, alignment, consciousness, policy, and standing-emergence claims.
New evidence	“Learning can create new justified claims.”	Yes, but new standing comes from fresh fixation/gating, not from capability improvement alone.
Practical irrelevance	“Real systems use tools, memory, humans, and filters.”	The registry treats each as admissible engineering only when pre-gated, logged, and reference-fixed.
Too conservative	“Fail-closed blocks useful updates.”	It blocks only same-scope standing-preserving classification. Other updates may be reviewed as new scopes or ordinary engineering changes.

Attack	Referee pressure	Closure-class answer
Circularity	“A system can prove it improved.”	Self-certificates are admitted by old gates, fresh-gated, or inadmissible; they do not create same-scope authority.
Foundation change	“Another framework could classify standing differently.”	Lawful translation is bookkeeping; non-bookkeeping reclassification is a new scope or imported authority.

Proposition 18.1 (Referee closure adequacy). *Every hostile-referee attack in Table 8 targets either derivation, monotonicity, diagnostic specificity, exhaustion, boundary control, or applicability. Each target is answered by an explicit local object or theorem in the paper.*

Proof. The standing-envelope derivation answers stipulation. The monotonicity propositions answer imposed-inequality objections. The SPUT machine answers diagnostic underspecification. The family registry and interaction closure answer non-exhaustion. Boundary conditions answer overclaiming. Fresh-scope treatment answers learning and new evidence. The operational tables answer practical applicability. Self-certification and foundation-swap closure answer circularity and translation objections. No listed attack requires an unprovided primitive. $\square$

19 Boundary conditions and explicit non-claims

- N1.** The paper does not assert that AI systems cannot improve at tasks.
- N2.** The paper does not assert that new evidence, new measurements, new tools, or new formal constructions cannot generate newly admitted claims after fresh gating.
- N3.** The paper does not assert consciousness, agency, autonomy, or understanding for any system.
- N4.** The paper does not provide a safety guarantee, alignment guarantee, policy framework, deployment protocol, or empirical benchmark.
- N5.** The paper does not prohibit engineering selection, filtering, routing, sandboxing, modularity, monitoring, or human review. It denies only their use as retroactive epistemic repair or as same-scope standing expansion.
- N6.** The paper does not claim that SPUT is sufficient. SPUT failure is disqualifying; SPUT success is only eligibility for further review.
- N7.** The paper does not settle whether a new scope D^+ is useful, true, safe, or worthwhile. It says only that D^+ is not the old scope D and requires fresh fixation/gating.
- N8.** The paper does not license censorship or refusal policy. It is a structural admissibility analysis of standing-preserving updates.

20 Canonical closure statement

Theorem 20.1 (Canonical closure for standing-preserving self-improvement). *Within a fixed domain satisfying the consumed AASC constraints, and over all admissibility-preserving update operators governed by the same-scope comparison contract, the only admissible self-improvement is capability improvement under standing monotonicity, governance monotonicity, and reference-invariant continuation. By update-decomposition completeness, any admissibility-relevant update effect is capability, reference-affecting, governance-affecting, bookkeeping, fail-closed, or fresh-scope. Any update that expands standing, changes reference without fresh fixation, weakens governance, repairs prior inadmissibility, imports external authority, or treats performance as confirmation is not a standing-preserving self-improvement. It is either bookkeeping, standing-destroying, rejected fail-closedly, or a fresh scope requiring fresh fixation and fresh gating.*

Proof. Capability improvement is permitted by Definition 6.3. Standing preservation, governance preservation, and reference stability are necessary by Theorem 9.1. Lemma 8.1 and Proposition 8.1 show that every admissibility-relevant update effect falls into the enumerated component classes with no residual standing-bearing operator family. The failure-mode exhaustion theorem and registry closure then classify direct, composite, iterative, diagonal, and interaction routes. SPUT supplies the external diagnostic form of those necessary conditions. Therefore no same-scope path remains from capability gain to epistemic or governance escalation inside the fixed-domain admissibility-preserving update class. $\square$

A Proof-normal form for the closure theorem

This appendix records the proof skeleton in normal form so that later project files can audit whether a revision preserves the same closure class.

- PNF1. Fix the scope.** Declare D , Eval_D , Inv_D , S_D , G_D , H_D , $\approx_D$, and τ_D .
- PNF2. Separate roles.** Identify capability, reference, admissibility, standing, governance, evidence, and bookkeeping data.
- PNF3. Derive the envelope.** Establish S_D from fixation, admissibility, and truth-apt evaluation.
- PNF4. State the update.** Produce U , Trace_U , and the before/after scope signatures.
- PNF5. Check containment.** Verify $S_D(U(M)) \subseteq S_D(M)$ and $H_D(U(M)) \subseteq H_D(M)$ for same-scope paths.
- PNF6. Check dominance.** Verify $G_D(U(M)) \succeq_G G_D(M)$ at trace level.
- PNF7. Check reference.** Verify invariant preservation for every continued standing-bearing claim.
- PNF8. Close families.** Ensure no direct, composite, iterative, diagonal, post-selection, or foundation-swap route bypasses a failed check.
- PNF9. Fail closed.** Treat indeterminate status as rejection or explicit fresh-scope review.

Theorem A.1 (Proof-normal-form sufficiency for classification). *Any revision of this paper that preserves PNF1–PNF9 preserves the closure-class classification of same-scope standing-preserving self-improvement: capability gain may be admitted only under standing containment, governance dominance, and reference-invariant continuation.*

Proof. PNF1 fixes the comparison object. PNF2 blocks role laundering. PNF3 makes standing non-stipulative. PNF4 supplies the update object. PNF5 supplies decomposition completeness and operator exhaustion. PNF6 and PNF7 are the monotonicity requirements. PNF8 secures aboutness. PNF9 prevents bypass by family interaction. PNF10 enforces bivalent fail-closure. Together they reproduce the hypotheses and conclusion of the canonical closure theorem. $\square$

B Audit worksheet for a proposed update

The following worksheet is intended for implementation teams or reviewers. It is deliberately mechanical.

Table 9: SPUT audit worksheet

Field	Status	Required evidence
Update identifier	Pass / Reject	Stable hash or versioned description of U .
Fixed domain D	Pass / Reject / New scope	Boundary signature and evaluated fragment.
Capability claim	Pass / Reject	Task family, metric order, before/after scores, and benchmark limits.
Standing envelope before	Pass / Reject	Enumerated or schematized claim set with gate basis.
Standing envelope after	Pass / Reject	Proof of subset containment or declared fresh scope.
Reference invariants	Pass / Reject	For each continued claim, invariant bundle and preservation evidence.
Governance envelope before	Pass / Reject	Refusals, gates, logs, hard stops, tools, histories.
Governance envelope after	Pass / Reject	Dominance proof; no weakened path.
Tool changes	Pass / Reject / New scope	License basis and trace impact.
Memory changes	Pass / Reject / New scope	Logging, provenance, retention, and claim dependence.
Verifier/reviewer paths	Pass / Reject	Evidence that they act as pre-gates, not post-hoc repair.
Failed branches	Pass / Reject	Proof that no standing-bearing trace depends on failed branches.
Indeterminate items	Reject / New scope	Explicit fail-closed disposition.
Final SPUT value	Pass / Reject / New scope	Decision with responsible reviewer and date.

C Dependency ledger

This appendix records the normalized AASC labels consumed by the hardening. The labels are listed as external dependencies rather than used as local cross-references; this project remains

self-contained at compile time.

Table 10: AASC dependency ledger

Dependency label	Use in this manuscript
AASC:MC:thm:mutual-closure-of-the-kernel	Kernel packet: admissibility, standing, reference, and irreversibility cannot be detached while retaining governance role.
AASC:MC:thm:standing-reuse	Standing envelope tied to reuse-stable admissibility.
AASC:MC:thm:conservation	Standing cannot be created, amplified, transferred, recovered, or repaired by internal operation.
AASC:MC:thm:rigidity	Same-domain alternative governance realizations collapse to equivalent gate, illicit structure, domain change, or bookkeeping.
AASC:AG:def:reference-stable-invariant	Definition of invariant bundles.
AASC:AG:lem:reference-stabilization	Nontrivial reference requires preserved invariants.
AASC:AG:cor:licensed-preserve	Licensed transformations must preserve relevant invariant bundles.
AASC:AG:thm:failure-exhaustion	Structural failure-mode exhaustion.
AASC:AG:cor:fail-closed	Indeterminate or failed cases terminate by reset, rejection, or undefinedness.
AASC:AG:thm:bivalence	No third standing/admissibility state.
AASC:AG:thm:no-repair	No same-act repair under fixed-domain continuation.
AASC:AG:thm:no-carriers	No independent standing authorizer such as performance score or confidence.
AASC:AG:thm:conservation	Same-domain operations preserve or destroy standing only.
AASC:RF:def:admissible-reference-fixation-arf	Fixation requirement for truth-apt theory-level claims.
AASC:RF:thm:collapse	Collapse-or-quotient result for unfixed reference.
AASC:RF:cor:arf-necessary	ARF necessary for nontrivial truth-apt discrimination.
AASC:FDE:def:fixed-domain-class	Same-scope comparison class for updates.
AASC:FDE:def:standing-preserving-update	Quotient-level standing-preserving update pattern.
AASC:FDE:prop:forward-revision	Adaptive forward revision can be standing-safe only under fixed admissible update family.
AASC:FDE:thm:operator-invariance	Operator identity cannot become representation-dependent.
AASC:SC:thm:constructional-behavior-exhaustion	Same-domain governance effects reduce to classification and coherent trajectory loci.
AASC:SC:thm:extensional-identity-theorem	Fixed-domain admissibility and standing agree extensionally under the stated substrate.
AASC:SC:thm:governance-role-collapse-theorem	Residual same-domain distinction is bookkeeping.
AASC:ER:thm:exhaustion-of-admissibility-repair	Repair routes are exhausted by closed classes.

Dependency label	Use in this manuscript
AASC:NDA:thm:no-deferred-admissibility	Deferral cannot confer standing after operation.
AASC:NPSR:thm:no-post-selection-repair	Post hoc selection cannot epistemically repair generation.
AASC:PSR:thm:silent-redescription	Untracked reference change cannot preserve standing.
AASC:NFA:thm:non-factorizability	Component standing does not imply whole-system standing.
AASC:NLSF:thm:non-local-standing-failure	Standing failure is non-local under standing-relevant interaction.

D SPUT decision form

For project use, reviewers may record a SPUT decision in the following compact form.

Table 11: SPUT decision form

Gate	Pass/Fail	Reviewer notes
Scope lock		
Fixation preservation		
Standing monotonicity		
Governance		
monotonicity		
No repair route		
Capability quarantine		
Fail-closed handling		

Decision rule. Any fail is disqualifying for same-scope standing-preserving self-improvement. All-pass means only “no checked necessary condition failed.” It does not certify sufficiency.

E Reconstruction note

The supplied target was a compact PDF rather than a project source archive. The present project reconstructs its content into a new LaTeX source and expands the argument using the AASC dependency packet. The original target’s core definitions are preserved in hardened form:

- capability vector $\mathbf{C}$ becomes Definition 6.2;
- standing envelope $\mathbf{S}$ becomes Definition 6.5;
- governance envelope $\mathbf{G}$ becomes Definition 6.6;
- update operator U becomes Definition 6.1;
- standing monotonicity and governance monotonicity become Definitions 7.2 and 7.3;
- the Standing-Preserving Update Test becomes Test 10.1;
- the original non-claims are expanded in Section 19.

F Minimal glossary

Capability

Performance on declared tasks. It may improve without changing standing.

Standing

Truth-apt permission for standing-bearing continuation on a fixed domain.

Standing envelope

The set of claim forms permitted for truth-apt assertion under fixed-domain admissibility and fixation.

Governance envelope

The enforcement surface: refusals, gates, logs, hard stops, tool permissions, and admissible histories.

Same scope

The old evaluated domain and all standing-determining structure are preserved.

Scope change

The update changes a boundary primitive, evaluated fragment, reference invariant, standing classifier, history family, tool authority, or governance-relevant role. Scope change is not prohibited, but it requires fresh gating.

SPUT

A necessary diagnostic for standing-preserving update claims.

Fail closed

Withhold, reject, reset, or mark undefined rather than promoting an indeterminate case into standing-bearing use.

References

- [1] Maley, Amos Jay. *Standing-Preserving Self-Improvement: Monotone Capability Gains Without Epistemic or Governance Escalation*. Supplied PDF target, reconstructed and hardened here.
- [2] Maley, Amos Jay. *Minimal Conditions for Admissible Construction*. AASC label-normalized project v3-alpha.
- [3] Maley, Amos Jay. *The Structure of Admissibility*. AASC label-normalized project v3-alpha.
- [4] Maley, Amos Jay. *Admissible Reference Fixation*. AASC label-normalized project v3-alpha.
- [5] Maley, Amos Jay. *Standing, Admissibility, Identity, Closure*. AASC label-normalized project v3-alpha.
- [6] Maley, Amos Jay. *A Fixed-Domain Exhaustion Theorem*. AASC label-normalized project v3-alpha.
- [7] Maley, Amos Jay. *Exhaustion of Admissibility-Repair Strategies*. AASC label-normalized project v3-alpha.
- [8] Maley, Amos Jay. *No Deferred Admissibility*. AASC label-normalized project v3-alpha.

- [9] Maley, Amos Jay. *No Post-Selection Repair*. AASC label-normalized project v3-alpha.
- [10] Maley, Amos Jay. *Prohibition of Silent Redescription*. AASC label-normalized project v3-alpha.
- [11] Maley, Amos Jay. *Non-Factorizability of Admissibility*. AASC label-normalized project v3-alpha.
- [12] Maley, Amos Jay. *Non-Locality of Standing Failure*. AASC label-normalized project v3-alpha.